PYTHON PROGRAMMING ASSIGNMENT REPORT

Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative

Course: CSA0806 – Python Programming

Student Name: Praveen L

Register Number: 1924260137

Department :AI AND DS

Faculty Name: HARIPRIYA

Submission Date: 23.10.26

Abstract

This report documents the analysis, design, implementation approach, and validation plan for a Python-based milk payment calculator for a village milk co-operative. The system accepts farmer and milk-quality data, calculates a rate from fat and solids-not-fat (SNF) readings, and computes the payment due. The assignment also considers numeric precision, input validation, functions, modules, algorithm tracing, testing, scaling, and a revised seasonal bonus rule. Evidence such as actual screenshots and measured benchmark results must be inserted by the student after running the submitted Python files.

Contents

- 1. Problem and Data Analysis
- 2. Numeric Types and Precision
- 3. Constants, Naming and Data Structures
- 4. Expressions and Rounding
- 5. Input Conversion and Validation
- 6. Function Design
- 7. Module Design
- 8. Algorithm, Pseudocode and UML
- 9. Implementation and Functional Testing
- 10. Performance and Scaling
- 11. Final Design Recommendation
- 12. Day-4 Design Challenge
- Appendix A: Evidence Checklist
- Appendix B: Submission Notes

System Overview and Requirements

The co-operative receives milk from approximately 300 farmers twice per day. The calculator uses milk quantity, fat percentage, SNF percentage, and shift information to calculate payment.

Parameter	Requirement / assumption
Base rate	₹30.00 per litre at 3.5% fat and 8.5% SNF
Fat adjustment	₹0.40 per litre for each 0.1 percentage-point difference from 3.5% fat
SNF adjustment	₹0.30 per litre for each 0.1 percentage-point difference from 8.5% SNF
Precision	Payment must be accurate to the nearest paisa
Scale	300, 6,000, and 150,000 farmers
Validation	Reject malformed and out-of-range inputs; use ranges specified by the assignment
Day-4 update	Apr–Jun base rate ₹32; fat step ₹0.45; >500 L in ten days earns ₹0.50/L bonus

Important interpretation: percentage-point steps are calculated relative to the stated reference values. The exact rounding policy (round each rate/payment or only the final total) must be stated consistently and checked against the instructor's specification.

Task 1 — Problem and Data Analysis

Inputs, outputs, constants and data types

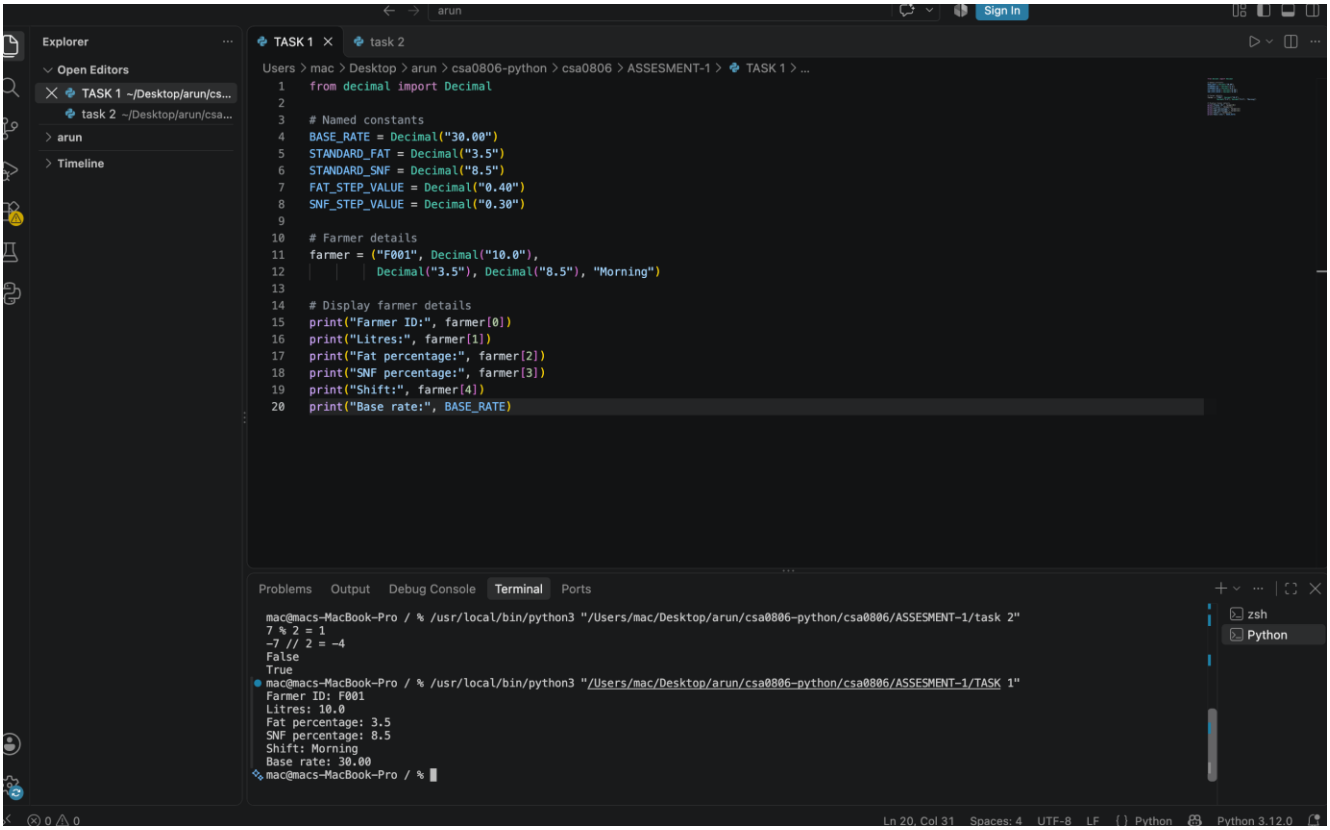
Item	Suggested type	Validation / purpose
Farmer ID	str	Non-empty; preserve leading zeros if present
Milk quantity (litres)	Decimal or float	Must be positive; define permitted upper limit
Fat percentage	Decimal or float	Numeric and within the permitted analyser range
SNF percentage	Decimal or float	Numeric and within the permitted analyser range
Shift	str or enum-like value	Accept only defined shift labels, e.g. morning/evening
Base rate	Decimal	₹30.00/L in the initial scenario
Rate adjustment steps	Decimal	₹0.40 per 0.1 fat point; ₹0.30 per 0.1 SNF point
Outputs	Decimal / formatted string	Rate per litre and total payout, formatted to two decimals

Assumptions

- The fat and SNF readings are percentages, not fractions.
- A step is based on a 0.1 percentage-point difference from the reference value.
- Farmer and shift identifiers are stored with each entry so that payments can be audited.
- Invalid data must be rejected with a useful message rather than silently accepted.
- Use a clearly documented rounding policy for currency.

Alternative data representations

Separate variables are straightforward for a single entry but become repetitive for many farmers. A tuple or dictionary groups the fields for one record; a list of records supports batches. A dictionary with named keys is readable and extensible. Processing n records takes O(n) time. Storing all records takes O(n) space; calculating a running total without retaining records takes O(1) extra space.



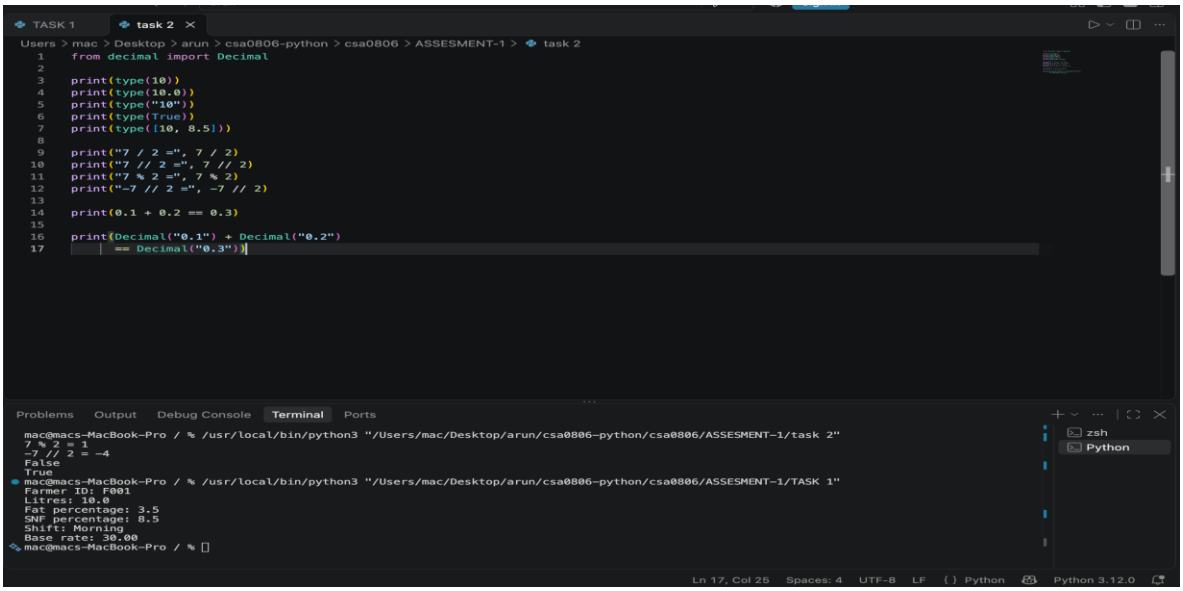
Task 2 — Numeric Types and Precision

Binary floating-point numbers cannot exactly represent many decimal fractions, which can cause small representation errors in values such as 0.1. Currency calculations should use Decimal constructed from strings, or integer paise, and apply a documented rounding rule.

```
from decimal import Decimal, ROUND_HALF_UP
amount = Decimal("12.345")
amount_paise = amount.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
```

Approach	Advantages	Limitations
float + round	Simple and usually fast	Binary representation can create edge-case surprises
Decimal	Base-10 decimal arithmetic and explicit rounding	Usually slower than float
Integer paise	Exact integer arithmetic for currency amounts	Intermediate rate calculations may need scaled integers

Evidence to insert: run a short demonstration comparing 0.1 + 0.2 with Decimal('0.1') + Decimal('0.2'), and paste the actual terminal output here.



Task 3 — Constants, Naming and Data Structures

BASE_RATE = Decimal("30.00")

REFERENCE_FAT = Decimal("3.5")

REFERENCE_SNF = Decimal("8.5")

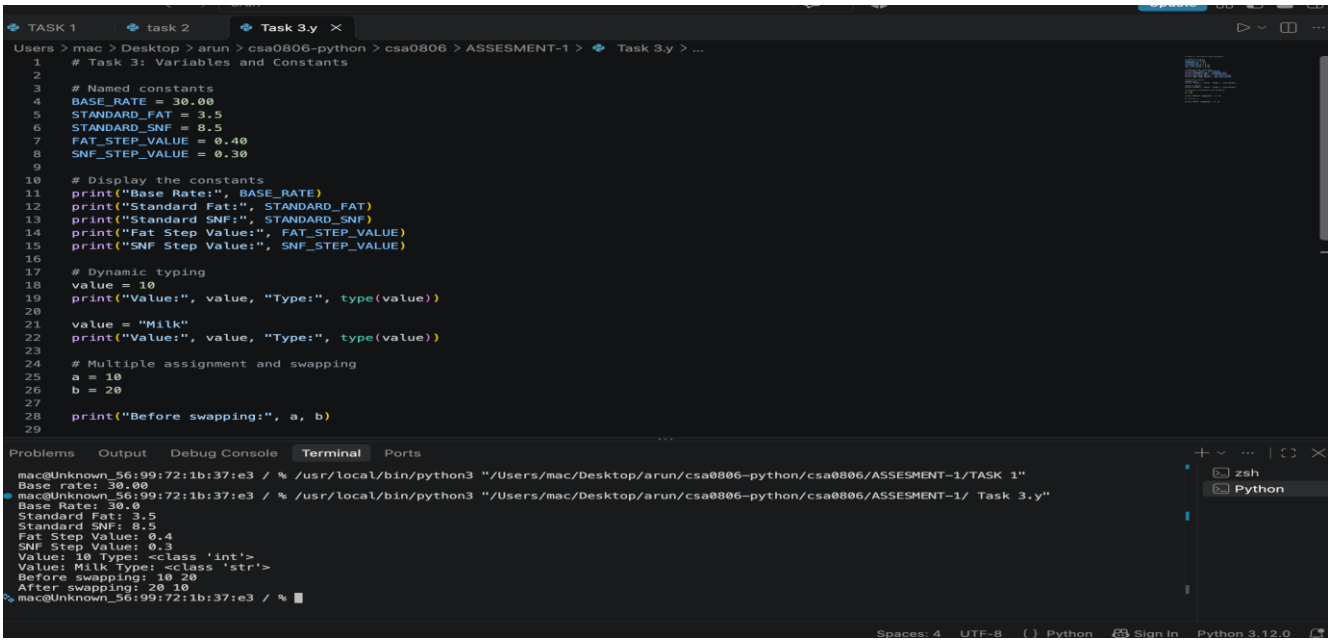
FAT_STEP_RATE = Decimal("0.40")

SNF_STEP_RATE = Decimal("0.30")

Named constants make policy visible and reduce errors caused by repeated magic numbers. Python is dynamically typed, so a variable can be rebound to a value of another type; this flexibility does not remove the need for clear names and validation. A configuration dictionary can group policy values.

RATES = {"base": Decimal("30.00"), "fat_step": Decimal("0.40"), "snf_step": Decimal("0.30")}

For swapping two variables in Python, use a, b = b, a. The assignment evaluates the right-hand side before assigning the two values.



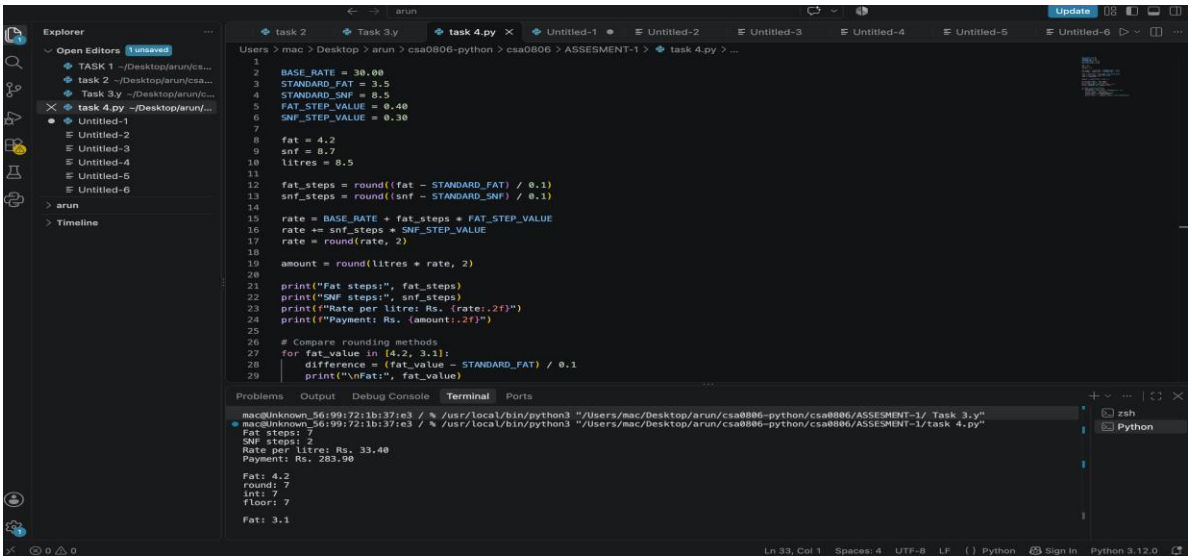
Task 4 — Expressions, Rate and Rounding

Rate model

```
fat_steps = (fat - REFERENCE_FAT) / Decimal('0.1')
snf_steps = (snf - REFERENCE_SNF) / Decimal('0.1')
rate = BASE_RATE + fat_steps * FAT_STEP_RATE + snf_steps * SNF_STEP_RATE
payout = litres * rate
```

This formula assumes the readings align with 0.1-point steps. If readings are not multiples of 0.1, the specification must clarify whether to round the number of steps or allow proportional adjustment. Do not use int() or floor division for currency unless truncation is explicitly required: they discard fractional parts. Format currency with two decimal places after applying the chosen rounding rule.

Operation	Effect
round(x, 2)	Rounds a numeric value to two decimal places using Python's built-in rounding semantics
int(x)	Truncates the fractional part toward zero
x // y	Floor division; rounds quotient down to an integer or lower multiple
Decimal.quantize(Decimal('0.01'), ROUND_HALF_UP)	Explicit two-decimal currency rounding



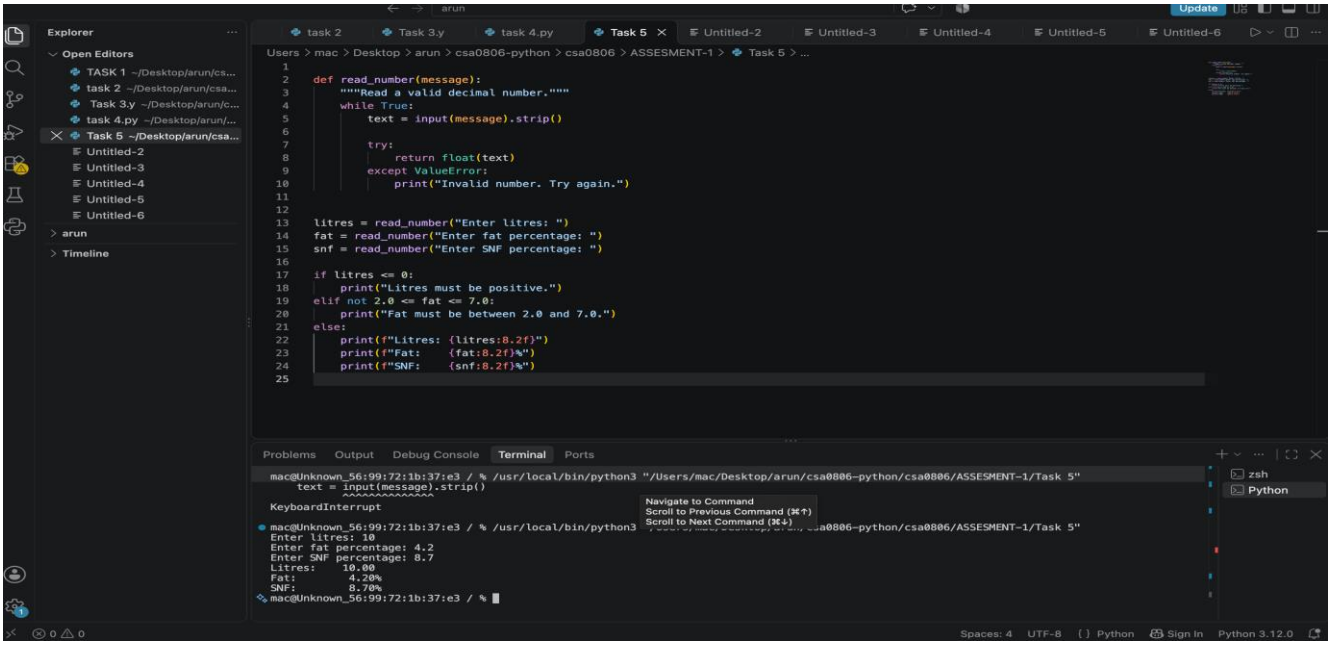
Task 5 — Input Conversion and Validation

input() returns a string. Convert numeric input only after stripping surrounding whitespace, and catch ValueError so text such as 'eight' or malformed decimal input such as '8,5' does not crash the program.

```
def read_number(prompt):
    while True:
        text = input(prompt).strip()
        try:
            return Decimal(text)
        except InvalidOperation:
            print('Invalid input. Enter a valid number.')
```

The exact valid ranges should match the assignment brief. Validate litres as positive, fat/SNF against the specified analyzer ranges, and shift against the permitted labels. Use f-strings such as f'{amount:.2f}' for display; for Decimal currency, quantize first and then format.

Evidence to insert: screenshots of one valid input, one malformed numeric input, and one out-of-range input.



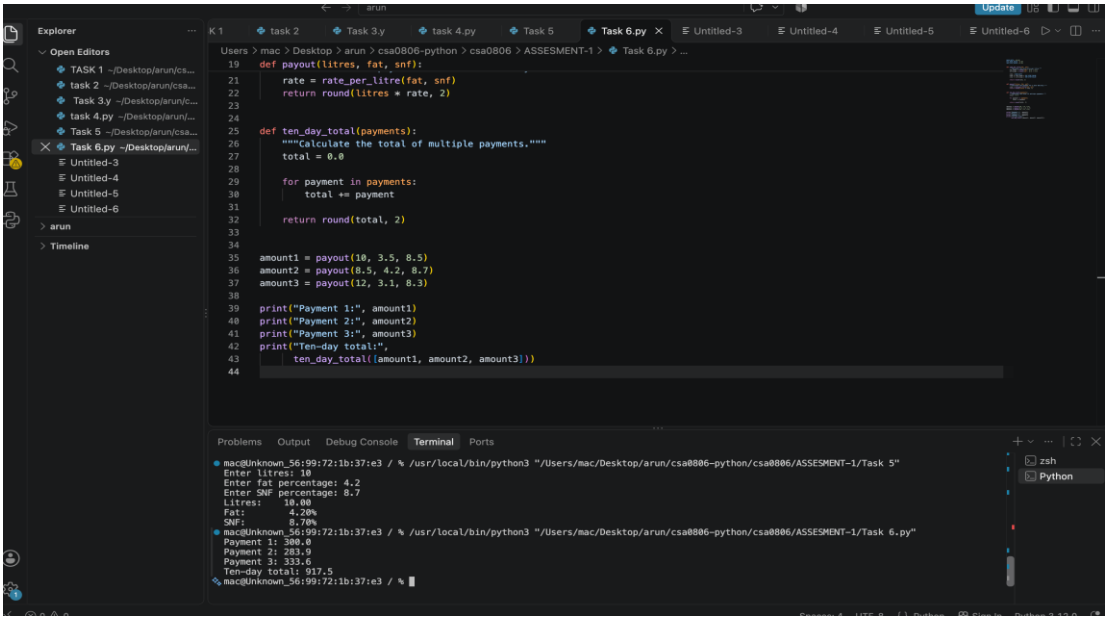
Task 6 — Function Design

Recommended responsibilities:

- rate_per_litre(fat, snf) returns the calculated rate and does not print.
- payout(litres, rate) returns the rounded payment amount.
- ten_day_total(entries) totals a sequence of entries and returns the result.
- Input/output code remains separate from calculation logic.

Returning values makes functions reusable in tests and other modules. Printing is a user-interface side effect and should be kept at the program boundary. Local variables are scoped to their function unless explicitly returned or otherwise shared. Use docstrings to describe parameters, return values, units, and rounding behavior.

```
def payout(litres, rate):
    """Return the payment for a milk quantity and per-litre rate."""
    return (litres * rate).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
```



Task 7 — Module Design

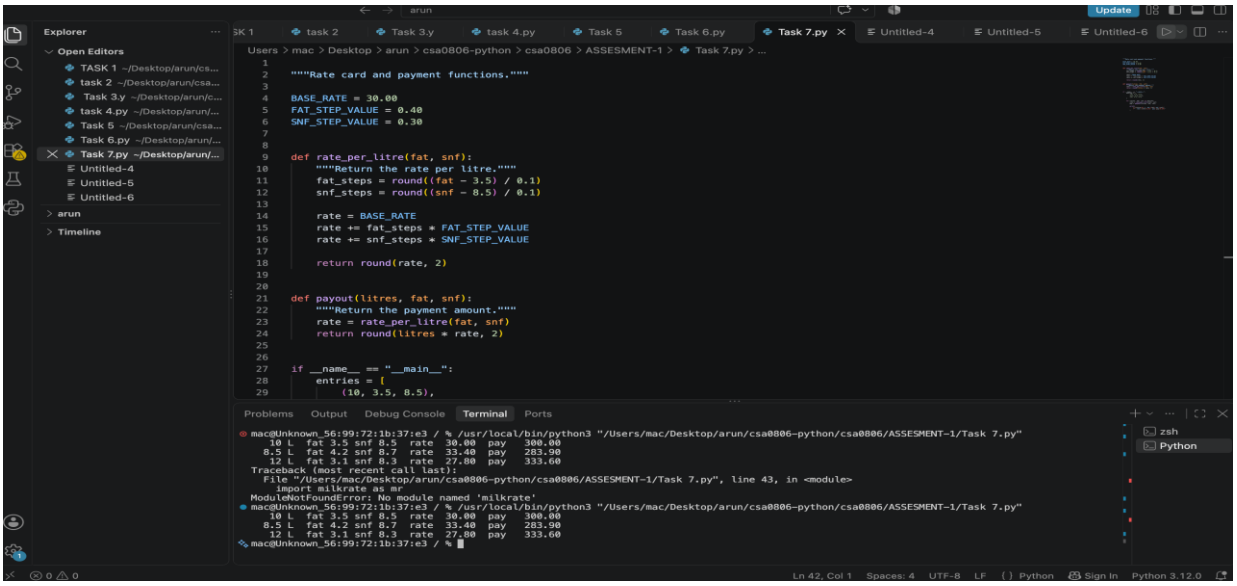
Place reusable calculation functions in milkrate.py and the interactive driver in a separate script. Both files should be in the same directory unless the project is packaged as a module.

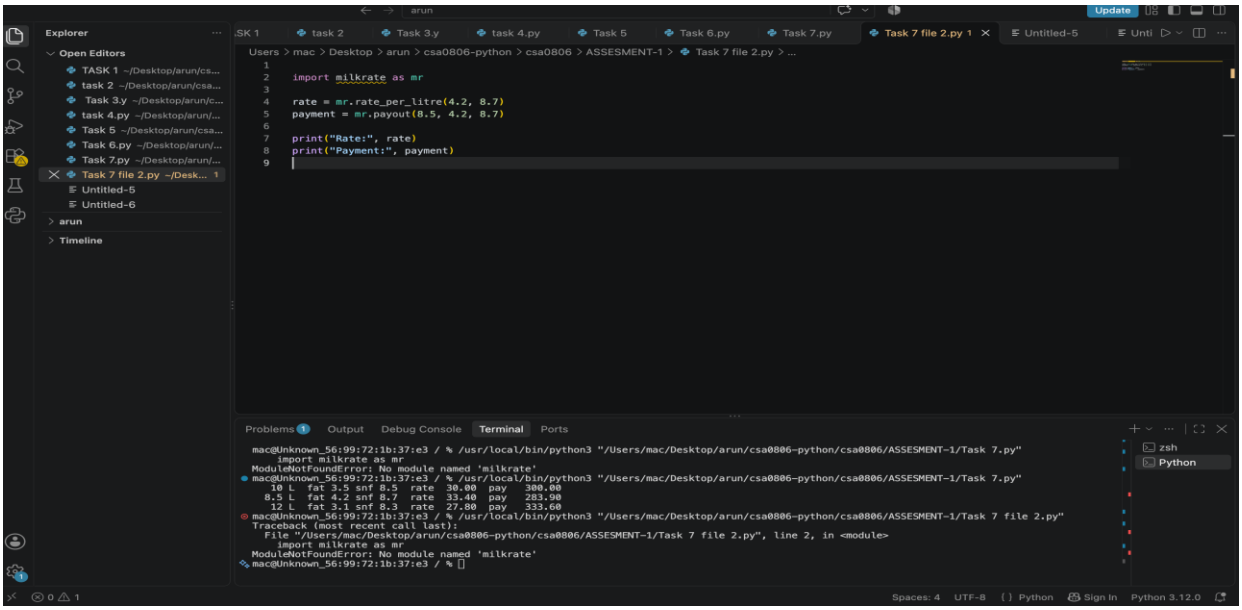
```
# milkrate.py
def rate_per_litre(fat, snf):
    # calculate and return the rate
    ...

# task7.py
from milkrate import rate_per_litre

if __name__ == "__main__":
    # read inputs and call imported functions
    ...
```

The if __name__ == '__main__' guard runs the driver only when the file is executed directly, not when imported. Replace the illustrative ellipses with the actual functions/code from the submitted files.





Task 8 — Algorithm, Pseudocode and UML

Pseudocode

START
Read farmer ID, litres, fat, SNF and shift.
Validate the input fields; if invalid, display an error and request valid data.
Calculate fat steps and SNF steps from the reference values.
Calculate rate per litre using the base rate and adjustments.
Calculate payout = litres × rate.
Round the payout according to the currency policy.
Display/store the farmer record and payout.
Repeat for remaining entries; calculate totals if required.
STOP

Text flowchart

[Start] -> [Read input] -> [Validate?]
| invalid -> [Show error / re-enter] -> [Read input]
| valid
v
[Calculate fat/SNF steps] -> [Calculate rate] -> [Calculate payout]
-> [Display/store result] -> [More entries?]
yes -> [Read input]
no -> [Display total] -> [End]

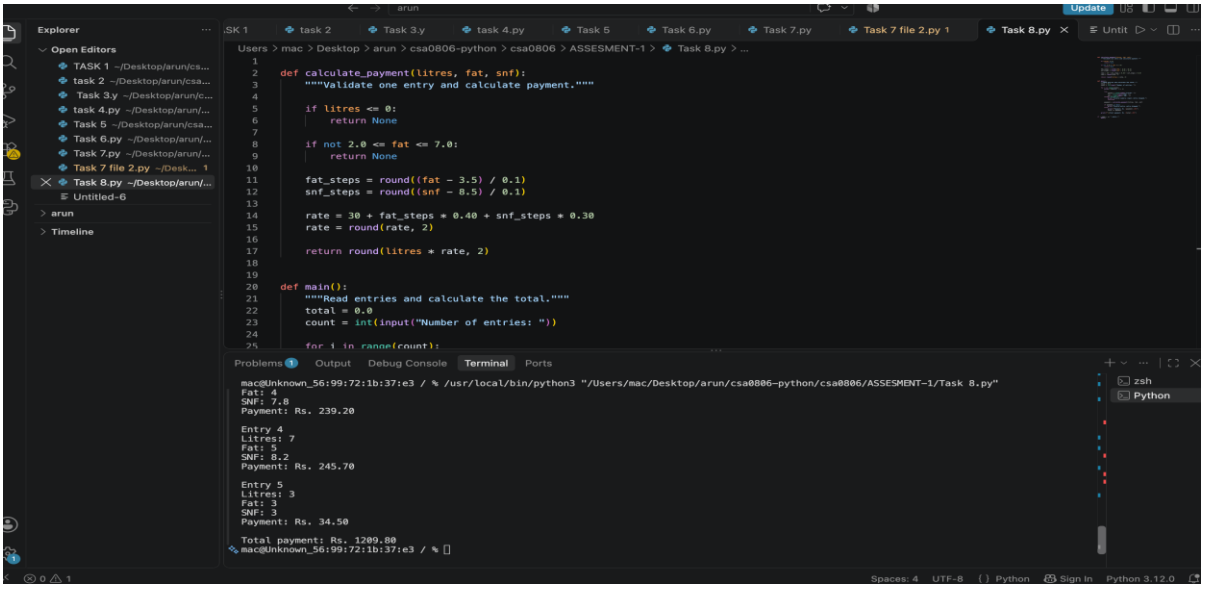
UML-style use case summary

Actor	Use case	Outcome
Co-operative operator	Enter farmer/milk details	Validated milk record
Co-operative operator	Calculate payment	Rate and payout
Co-operative operator	Review batch total	Total payment for selected entries
Developer/tester	Run test cases	Pass/fail evidence

UML-style class/module view

Operator -> MilkPaymentCalculator
MilkPaymentCalculator
+ rate_per_litre(fat, snf) : Decimal
+ payout(litres, rate) : Decimal
+ ten_day_total(entries) : Decimal

This is a conceptual UML-style view. If formal UML notation is required, recreate it using the standard actor/use-case and class diagram symbols and paste the diagram image into this section.

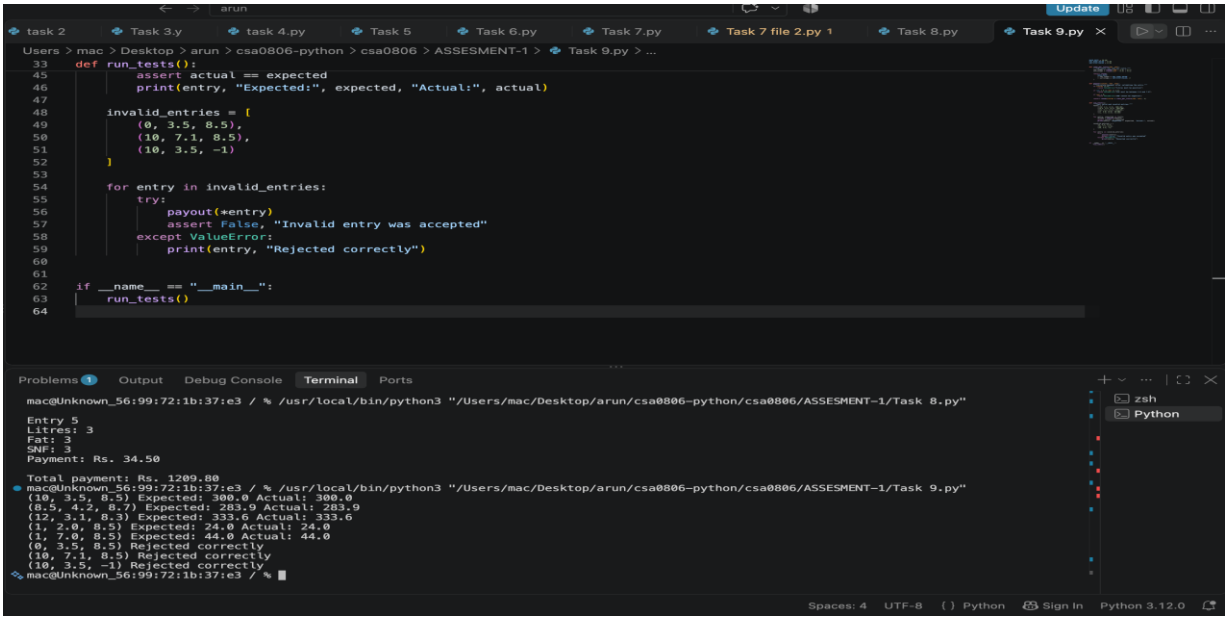


Task 9 — Full Calculator and Functional Tests

The integrated program should connect input validation, rate calculation, payout calculation, and result display. Use the exact submitted code as the implementation evidence; do not paste pseudocode in place of a required source listing.

Test ID	Test purpose	Input	Expected result / check	Actual result
T09-01	Normal case	10 L, 4.2% fat, 8.7% SNF	Valid rate and payout; two-decimal currency	Fill after run
T09-02	Reference values	10 L, 3.5% fat, 8.5% SNF	Rate = ₹30.00/L; payout = ₹300.00	Fill after run
T09-03	Zero litres	0 L	Rejected if quantity must be positive	Fill after run
T09-04	Negative litres	-1 L	Rejected	Fill after run
T09-05	Malformed input	'eight' litres	Validation message; no unhandled crash	Fill after run
T09-06	Five records	Five valid entries	Each record processed; total equals sum of record payouts	Fill after run

Do not mark a test as passed until the program has actually been run and the observed output has been recorded.

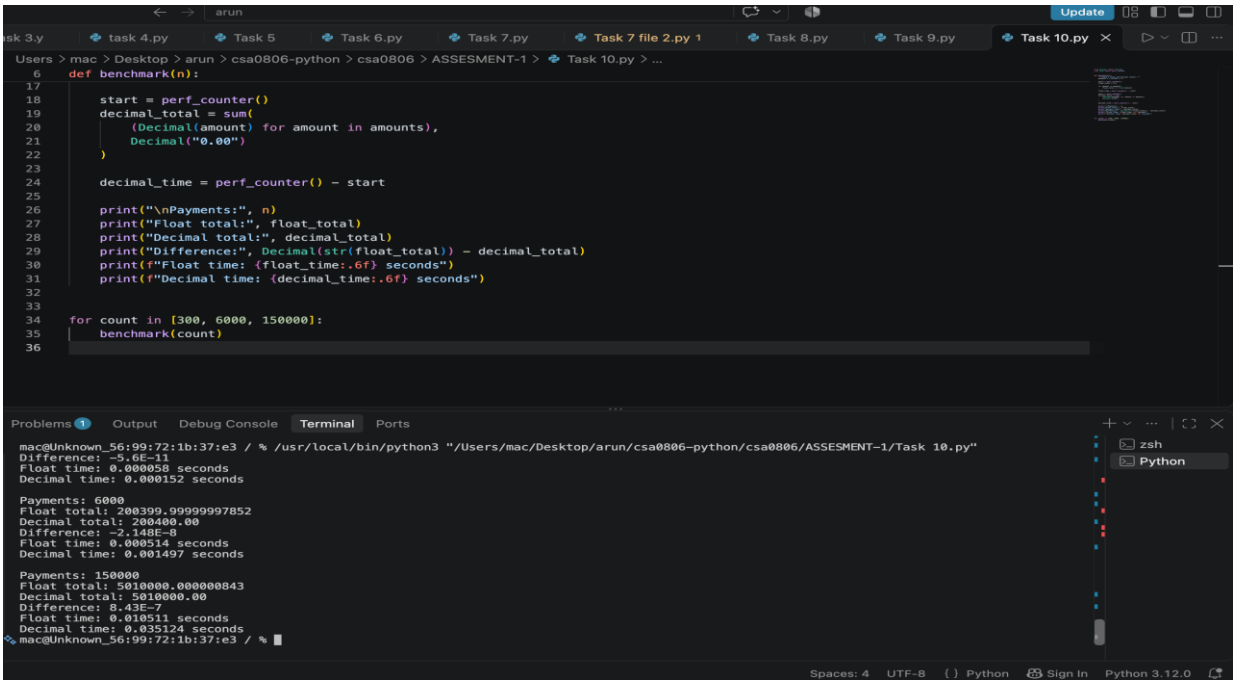


Task 10 — Performance, Scaling and Numeric Comparison

Benchmark float and Decimal implementations with the same generated dataset and equivalent rounding policy. Use time.perf_counter() around only the calculation loop, repeat the measurement, and report the median or average. Do not invent benchmark values: measure them on the target machine.

Number of farmers	Records per day at 2 shifts	Expected complexity for one pass	Measured time
300	600	O(n)	Fill after run
6,000	12,000	O(n)	Fill after run
150,000	300,000	O(n)	Fill after run

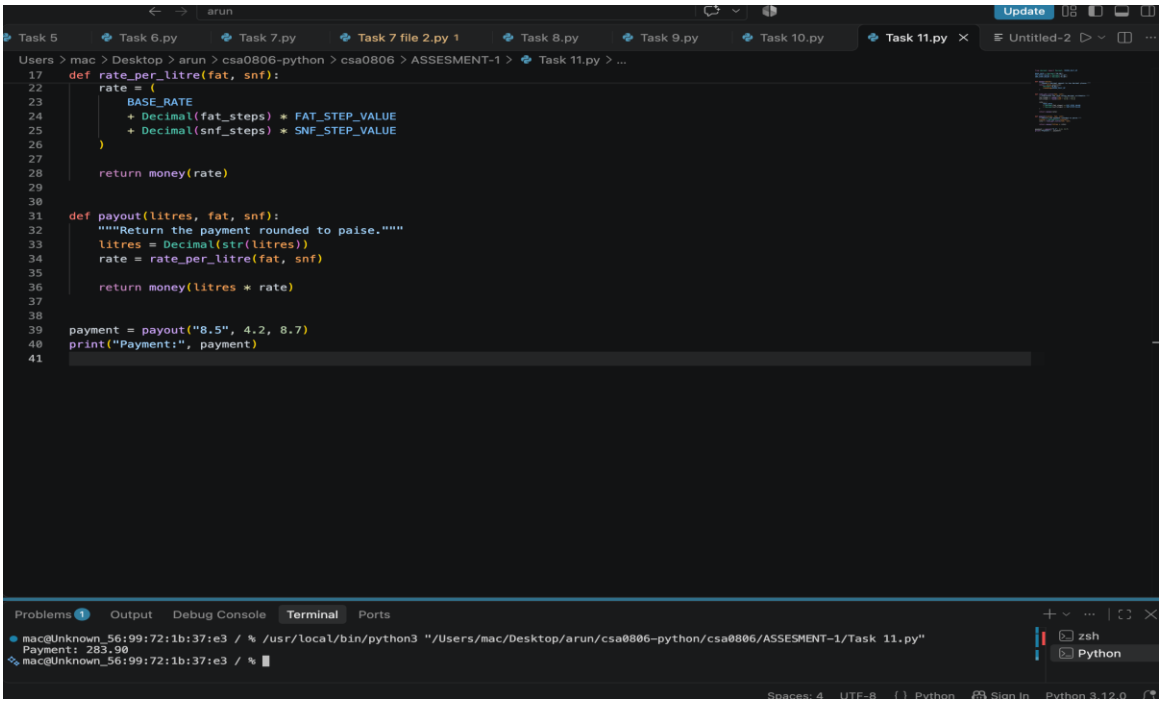
For n records, a single calculation pass is O(n) time. Retaining every record requires O(n) storage; a streaming total requires O(1) extra storage. If using a cache/dictionary, explain the key, invalidation rule, and whether the calculation is repeated often enough to justify it.



Task 11 — Final Design Recommendation

Design decision	Recommendation	Reason
Currency arithmetic	Decimal or integer paise	Avoid binary float representation issues in money
Policy values	Named constants/configuration	Avoid magic numbers; simplify policy changes
Code organization	Small functions in a reusable module	Testable, readable and maintainable
Data storage	List of records for batch reporting; stream for very large totals	Choose based on audit/reporting needs
Validation	Centralized validation functions	Consistent errors and data quality
Testing	Normal, boundary, invalid, and batch cases	Covers typical and failure behavior

The final choice should be supported by actual test output and the benchmark measurements from Task 10. If policy rates change by season, load the rate configuration from a clearly documented dictionary or configuration file rather than editing formulas in multiple places.



Task 12 — Day-4 Design Challenge

Updated business rules

- For April through June, the base rate is ₹32.00 per litre.
- Each 0.1 percentage-point fat step changes the rate by ₹0.45 per litre.
- A farmer supplying more than 500 litres over a ten-day period receives a bonus of ₹0.50 per litre.
- The original SNF adjustment is not explicitly stated as removed; confirm whether it remains active in the updated rules.

Algorithm

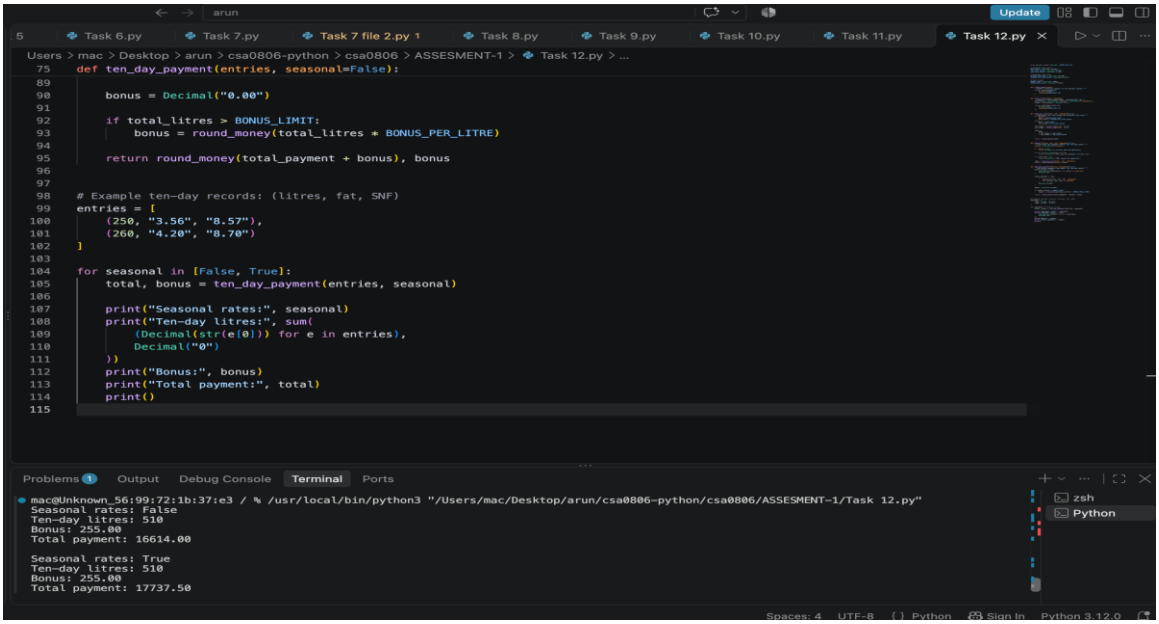
Collect each farmer's records over a ten-day window.
Sum litres for each farmer over that window.
Select the applicable seasonal base rate and fat adjustment.
Calculate the normal payout using the confirmed active quality adjustments.
If the farmer's ten-day total is greater than 500 litres, add ₹0.50 per eligible litre, according to the clarified bonus policy.
Round and report the final amount; retain the calculation breakdown for audit.

Challenge test table

Test ID	Ten-day total	Bonus eligibility	Expected check
T12-01	500.00 L	No	Strictly greater than 500 is not met
T12-02	500.01 L	Yes	Threshold exceeded
T12-03	501.00 L	Yes	Bonus applies per confirmed policy
T12-04	Invalid/missing record	Cannot determine	Reject or flag incomplete data

Policy clarification required: the brief says farmers supplying more than 500 litres in ten days receive ₹0.50/L bonus. Confirm whether the bonus is applied to all litres supplied in the ten-day period or only litres above 500.

Also confirm whether SNF adjustment remains active and how records crossing a month boundary are assigned to a season.



Conclusion

The proposed calculator separates data collection, validation, rate computation, and reporting. Decimal or integer-based currency arithmetic, reusable functions, clear constants, modular code, and systematic testing improve correctness and maintainability.